

4. Auditoría de seguridad del API mediante npm audit

Auditoría inicial

El primer paso consistió en ejecutar el siguiente comando desde la carpeta raíz del proyecto:

npm audit

```
PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> npm audit
node_modules/serve-static

11 vulnerabilities (4 low, 2 moderate, 4 high, 1 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force
PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> 
```

Este comando permitió revisar todas las dependencias instaladas y compararlas con la base de datos de vulnerabilidades mantenida por npm.

Resultado obtenido

La auditoría inicial reportó el siguiente estado:

Nivel de severidad	Cantidad
Low	4
Moderate	2
High	4
Critical	1
Total	11 vulnerabilidades

Las principales dependencias afectadas fueron:

- body-parser
- express
- cookie

- mongodb
- morgan
- on-headers
- path-to-regexp
- qs
- send

El reporte también indicó que varias de estas vulnerabilidades podían corregirse actualizando las dependencias del proyecto a versiones más recientes.

Análisis de las vulnerabilidades encontradas

Después de revisar el reporte generado por npm, se identificó el riesgo asociado a cada una de las principales librerías.

body-parser

Presentaba una vulnerabilidad de tipo Denegación de Servicio (DoS) al procesar solicitudes codificadas mediante URL.

Implicación

Un atacante podría enviar solicitudes especialmente diseñadas para consumir recursos del servidor y afectar la disponibilidad del servicio.

cookie

Permitía el uso de caracteres inválidos dentro del nombre y atributos de las cookies.

Implicación

Esta situación podría ocasionar problemas durante la administración de sesiones y el manejo de cookies.

mongodb

El controlador de MongoDB podía registrar información relacionada con la autenticación.

Implicación

Existía la posibilidad de exponer información sensible dentro de los registros del servidor.

morgan

Presentaba una vulnerabilidad conocida como Log Forging.

Implicación

Un atacante podría alterar el contenido de los registros de actividad para ocultar acciones maliciosas.

on-headers

Permitía la manipulación de encabezados HTTP.

Implicación

Podía afectar la integridad de las respuestas generadas por el servidor.

path-to-regexp

Presentaba vulnerabilidades de tipo Regular Expression Denial of Service (ReDoS).

Implicación

Una solicitud especialmente diseñada podía consumir excesivamente el procesador y ralentizar el funcionamiento del API.

qs

Presentaba vulnerabilidades relacionadas con Prototype Pollution y ataques de Denegación de Servicio.

Implicación

Un atacante podía modificar objetos internos de JavaScript o provocar un consumo elevado de memoria.

send

Presentaba una vulnerabilidad que podía facilitar ataques XSS al servir determinados archivos.

Implicación

En ciertos escenarios un atacante podría intentar inyectar contenido malicioso.

Primera corrección automática

Como primera medida se ejecutó el siguiente comando recomendado por npm:

```
npm audit fix
```

Este proceso actualizó automáticamente aquellas dependencias que podían corregirse sin modificar la compatibilidad del proyecto.

```
PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> npm audit fix
node_modules/serve-static

9 vulnerabilities (4 low, 1 moderate, 4 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force
❖ PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> 
```

Resultado obtenido

Después de ejecutar el comando se realizó nuevamente la auditoría.

npm audit

```
PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> npm audit
node_modules/serve-static

9 vulnerabilities (4 low, 1 moderate, 4 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force
❖ PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> 
```

El nuevo resultado fue:

Nivel de severidad	Cantidad
Low	4
Moderate	1
High	4
Critical	0
Total	9 vulnerabilidades

Análisis

Se observó una mejora importante, ya que la vulnerabilidad clasificada como Critical fue eliminada y el número total de vulnerabilidades disminuyó de 11 a 9.

Sin embargo, el propio reporte indicó que todavía existían dependencias antiguas de Express y Morgan que impedían eliminar completamente los problemas de seguridad.

Revisión de las versiones instaladas

Para identificar el origen de las vulnerabilidades restantes se verificó la versión de las principales dependencias mediante los siguientes comandos:

```
npm list express
```

```
npm list morgan
```

```
npm list mongoose
```

Además, se revisó el archivo package.json.

Se encontró que el proyecto utilizaba versiones relativamente antiguas.

Dependencia Versión anterior

Express 4.16.4

Morgan 1.9.1

Body-parser 1.19.1

Mongoose 6.13.10

Debido a ello se decidió realizar una actualización manual de las librerías principales.

Actualización manual de dependencias

En lugar de utilizar inmediatamente `npm audit fix --force`, se optó por actualizar primero las dependencias de forma controlada.

Se ejecutaron los siguientes comandos:

```
npm install express@4.22.2
```

Durante esta actualización npm reemplazó varias dependencias internas del proyecto.

Después se actualizó Morgan.

```
npm install morgan@1.11.0
```

Al finalizar esta actualización npm indicó temporalmente que el proyecto no presentaba vulnerabilidades.

Posteriormente se actualizó Body Parser.

```
npm install body-parser@1.20.3
```

Al final se ejecutó:

```
npm prune
```

con el objetivo de eliminar dependencias que ya no eran utilizadas por el proyecto.

Segunda auditoría

Después de realizar las actualizaciones manuales se ejecutó nuevamente:

```
npm audit
```

```
PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> npm audit
fix available via `npm audit fix`
node_modules/body-parser/node_modules/qs
  body-parser 1.20.3 - 1.20.4 || 2.0.0-beta.1 - 2.0.2
  Depends on vulnerable versions of qs
  node_modules/body-parser

2 moderate severity vulnerabilities

To address all issues, run:
  npm audit fix
PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress>
```

En esta ocasión únicamente permanecían dos vulnerabilidades de severidad Moderate, ambas relacionadas con la dependencia qs, utilizada internamente por body-parser.

El propio reporte recomendó ejecutar:

```
npm audit fix
```

para completar la actualización.

Corrección final

Se ejecutó nuevamente:

```
npm audit fix
```

```

• PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> npm audit fix

removed 3 packages, changed 6 packages, and audited 163 packages in 1s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
❖ PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> 

```

Una vez finalizado el proceso se realizó la última auditoría.

npm audit

```

• PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> npm audit fix

removed 3 packages, changed 6 packages, and audited 163 packages in 1s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
• PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> npm audit
found 0 vulnerabilities
❖ PS C:\Users\madmu\Downloads\Practica-03\Practica-03\03b - ArticleExpress> 

```

El resultado obtenido fue:

found 0 vulnerabilities

Comparación de resultados

Etapas	Vulnerabilidades
Auditoría inicial	11
Después de npm audit fix	9
Después de actualizar Express, Morgan y Body Parser 2	
Después del último npm audit fix	0

Medidas aplicadas

Durante este proceso se realizaron las siguientes acciones para mejorar la seguridad del proyecto:

- Auditoría completa de todas las dependencias mediante npm audit.

- Actualización de Express a la versión 4.22.2.
- Actualización de Morgan a la versión 1.11.0.
- Actualización de Body Parser a la versión 1.20.3.
- Eliminación de dependencias innecesarias mediante npm prune.
- Nueva auditoría para verificar los resultados.
- Corrección final mediante npm audit fix.